Incremental Logic Explanation

Incremental processing ensures that only new or changed data is processed, improving performance, scalability, and reliability.

1. Why Incremental Processing Is Required

Incremental processing is essential to:

- Avoid full data reloads
- Reduce compute cost
- Enable frequent pipeline runs
- Prevent data duplication
- Support enterprise-scale data volumes

2. Incremental Strategy Overview

The pipeline implements incremental logic at **multiple layers**, based on the responsibility of each layer:

Layer	Incremental Strategy
Bronze	File-based incremental ingestion
Silver	MERGE-based incremental UPSERT
Gold	Full refresh from Silver

3. Bronze Layer – Incremental Ingestion

Approach

- Raw data files are ingested from Azure Data Lake Storage
- Only **new files** dropped into the raw directory are processed
- Previously ingested files are not reprocessed

Key Characteristics

- Append-only ingestion
- Partitioned by ingestion date
- Original schema preserved

Benefits

- Prevents duplicate ingestion
- Simplifies raw data handling
- Supports high-volume ingestion

4. Silver Layer – Incremental Processing

Approach

- Silver tables are updated using **Delta Lake MERGE**
- Natural business keys (e.g., transaction_id) are used for matching
- Records are either:
  - Updated if they already exist
  - Inserted if they are new

Example Logic

```
MERGE INTO silver_table t
USING incoming_data s
ON t.business_key = s.business_key
WHEN MATCHED THEN UPDATE
WHEN NOT MATCHED THEN INSERT
```

Benefits

- Idempotent pipeline execution
- Safe reruns without duplication
- Supports late-arriving data

5. Watermark Concept

Although file-based ingestion is used, the source data includes a last_updated_ts column which can be used as a watermark for:

- Detecting updated records
- Handling late-arriving data
- Supporting future streaming extensions

6. Gold Layer – Refresh Strategy

Approach

- Gold tables are derived from validated Silver data
- Full refresh is performed for simplicity and consistency
- Aggregations are optimized and relatively lightweight

7. Idempotency and Rerun Safety

The pipeline is designed to be **idempotent**, meaning:

- Multiple executions produce the same result
- No duplicate records are introduced
- Partial failures do not corrupt data

This is achieved using:

- Append-only Bronze ingestion
- MERGE-based Silver processing
- ACID guarantees of Delta Lake